



RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT - 2

Prepared by: Mohamed. B. Sirajuddeen

Reg No : 22TD0053

Subject Name : Embedded Systems

Subject Code : CST62

Year/Sem/Sec : 43/56/B

Date of Submission: 9/4/2025

Unit : 2

Teacher's Sign :

Department Vision And Mission

Vision:

Cultivate student as technocrats researchers and entrepreneurs in the field of computer science and engineering.

Mission:

- * The impact quality education in computer science and engineering by means of learning techniques and value-added courses.
- * To inculcate professional excellence and research culture by encouraging projects in cutting edge techniques industry interaction.
- * To build leadership, skills ethical values and team works among the students.
- * To strengthen the collaboration of departments, and industry internships mentorships and professional body activities.

1. Explain the format of a typical data processing instruction in ARM. Describe the role of each field in the instruction. Give two examples and explain what they do.

The data processing instructions manipulate data within registers. They are move instructions, arithmetic instructions, logical instructions, comparison instructions, and multiply instructions. Most data processing instructions can process one of their operands using the barrel shifter.

If you use the S suffix on a data processing instruction, then it updates the flags in a CPSR.

Move and logical operations update the carry flag C, negative flag N, and zero flag Z.

The carry flag is set from the result of the barrel shift as the last bit shifted out.

The N flag is set if bit 31 of the result is 1. The Z flag is set if the result is zero.

Typical format of a data processing instruction:

1. <opcode> - operation code (eg: ADD, SUB, AND, ORR, MOV, etc)
2. <Rd> - Destination register where the result is stored
3. <Rn> - First operand register
4. <operand 2> - Second operand (which can be a register or an immediate value)
5. <Dest> - The destination register or memory location where the result will be stored
6. <SRC1> - The first source register or memory location ~~optional~~

7. $\langle \text{SRC2} \rangle$ - The second source register or memory location (optional)

MOVE Instruction

- Move is the simplest ARM instruction.
- It copies N into a destination register Rd , when N is a register or immediate value.
- This instruction is useful for setting initial values and transferring data between registers.

Syntax : $\langle \text{instruction} \rangle \{ \langle \text{cond} \rangle \} \{ S \} Rd, N$

MOV	Move a 32-bit value into a register	$Rd = N$
MOVN	Move the NOT of the 32-bit value into a register	$Rd = \sim N$

MOV Move a 32-bit value into a register.

MOV $\langle \text{cond} \rangle \{ S \}, Rd, \# \langle \text{rotated_immed} \rangle$
 $Rd = \langle \text{rotated_immed} \rangle$

Example The MOV instruction takes the contents of register $r5$ and copies from into register $r7$. In this case, taking the value 5, and the value 8 in register $r7$.

PRE:

$$r5 = 5$$

$$r7 = 8$$

MOV $r7, r5$; let $r7 = r5$

POST:

$$r5 = 5$$

$$r7 = 5$$

Arithmetic Instructions:

The arithmetic instructions implement addition and subtraction of 32-bit signed and unsigned values.

Syntax

<instruction> {<cond>} {S} Rd, Rn, N

ADC	add two 32-bit values and carry	$Rd = Rn + N + \text{carry}$
ADD	add two 32-bit values	$Rd = Rn + N$
RSB	Reverse subtract of two 32-bit values	$Rd = N - Rn$
RSC	Reverse subtract with carry of two 32-bit values	$Rd = N - Rn - 1$ (carry flag)
SBC	Subtract with carry of two 32-bit values	$Rd = Rn - N - 1$ (carry flag)
SUB	Subtract two 32-bit values	$Rd = Rn - N$

ADD: N is the result of the shifter operation.

Add two 32-bit values

ADD <cond> {S} Rd, Rn, # <rotated-immed>

$Rd = Rn + \text{<rotated-immed>}$ CPSR

updated if S suffix specified

Eg:

(i) ADD r0, r1, #4 ; $r0 = r1 + 4$

(ii) ADDS r0, r2, r2 ; $r0 = r2 + r2$ and flags updated

ADC:

Add two 32-bit values and carry

ADC <cond> {S} Rd, Rn, # <rotated-immed>

$Rd = Rn + \text{<rotated-immed>}$ + CPSR updated

Eg: ADC r1, r1, r3

SUB: Subtract instruction subtracts a value stored in register r2 from a value stored in register r1. The result is stored in register r0.

Example

PRE

$r_0 = 0x00000000$

$r_1 = 0x00000002$

$r_2 = 0x00000001$

SUB r_0, r_1, r_2

POST

$r_0 = 0x00000000$

RSB: Reverse subtract instruction subtract r_1 from the constant value #0, writing the result to r_0 .

Example

PRE

$r_0 = 0x00000000$

$r_1 = 0x00000077$

RSB $r_0, r_1, \#0$; $Rd = 0x0 - r_1$

POST

$r_0 = -r_1 = 0x\text{fff fff 89}$

SUBS: The SUBS instruction is useful for decrementing loop counters. In the example we subtract the immediate value one from the value one stored in register r_1 .

Example

PRE

CPSR = NZCVg; FT - USER

$r_1 = 0x00000001$

$r_1 \wedge r_1, \#1$

SUBS

POST

CPSR = NZCVg; FT - USER

$r_1 = 0x00000000$

SBC: Subtract with carry

$SBC \langle cond \rangle \{s\}, Rd, Rn, \# \langle rotated_immed \rangle$

$Rd = Rn - \langle rotated_immed \rangle$

Eg: $SBC\ r1, r1, r3$; subtract high words and borrow.

Logical Instructions

Logical instructions perform bitwise logical operations on the two source registers.

Syntax: $\langle instruction \rangle \{ \langle cond \rangle \} \{s\}, Rd, Rn, N$

AND	logical bitwise AND of two 32-bit values	$Rd = Rn \& N$
ORR	Logical bitwise OR of two 32-bit values	$Rd = Rn N$
EOR	logical exclusive OR of two 32-bit values	$Rd = Rn \wedge N$
BIC	Logical bit clear (AND NOT)	$Rd = Rn \sim N$

AND: Logical bitwise AND of two 32-bit values

Examples

$AND\ r0, r0, \# 0xFF$

• $r0 = r0 \& 0xFF$

$ANDS\ r0, r0, r1$

• $r0 = r0 \& r1$

• CPSR updated

ORR

Logical OR operation between register $r1$ and $r2$, $r0$ holds the result.

Example

PRE: $r0 = 0x00000000, r1 = 0x02040608,$

$r2 = 0x10305070$

ORR $r0, r1, r2$

POST $r0 = 0x12345678$

EOR Logical exclusive OR of two 32-bit values

(i) EOR <cond> {s} Rd, Rn, # <rotated-immed>

(ii) EOR <cond> {s} Rd, Rn, Rm {<shift>} $Rd = Rn$
 $Rd = Rn$

BIC Logical bit clear (AND NOT) of two 32-bit values.

(i) BIC <cond> {s} Rd, Rn, # <rotated-immed>

eg: PRE $r1 = 0b1111$ BIC $r0, r1, r2$
 $r2 = 0b0101$

POST $r0 = 0b1010$

Comparison Instructions

The comparison instructions are used to compare or test a register with a 32-bit value. They update the cpsr flag bits according to the result.

Syntax: <instruction> {<cond>} Rn, N

CMN	Compare Negated	flags set as a result $Rn + N$
CMP	Compare	flags set as a result $Rn - N$
TEQ	Test for equality of 32-bit values	flags set as a result $Rn \wedge N$
TST	Test bits of a 32-bit value	flags set as a result $Rn \wedge N$

CMP

(i) CMP <cond> Rn, # <rotated-immed>

(ii) CMP <cond> Rn, Rm {<shift>}

CMN

- (i) CMN <cond> Rn, # <rotated-immed>
- (ii) CMN <cond> Rn, Rm {, shift}

TST

Rn, # <rotated-immed>

TST r0, # 0xFF

TEQ

TEQ <cond> Rn, # <rotated-immed>

TEQ r0, #1;

Multiply Instruction

The multiply instructions multiply the contents of a pair of registers and depending upon the instruction, accumulate the results in with another register.

Syntax MLA {<cond>} {s} Rd, Rm, Rs, Rn

MUL {<cond>} {s} Rd, Rm, Rs

MLA	multiply and accumulate	$Rd = (Rm * Rs) + Rn$
MUL	multiply	$Rd = Rm * Rs$

Example

PRE

r0 = 0x00000000

r1 = 0x00000002

r2 = 0x00000002

MUL # r0, r1, r2 ; r0 = r1 * r2

POST

r0 = 0x00000004

r1 = 0x00000002

r2 = 0x00000002

Branch Instructions

A branch instruction changes the flow of execution is used to call a routine.

Syntax

B { <cond> } label

BL { <cond> } label

BX { <cond> } Rn

BLX { <cond> } label | Rm

B	Branch	PC = label
BL	branch with link	PC = label br = address of the next inst after the BL.
BX	branch exchange	PC = Rm & 0xFFFFFFFF, T = Rm & 1
BLX	Branch exchange with link	PC = label, T = 1 PC = Rm & 0xFFFFFFFF, T = Rm & 1 br = addr of the next inst after the BLX.

Example

B forward ADD

r1, r2, #4 ADD

r0, r6, #2 ADD

r3, r7, #4

forward

SUB r1, r2, #4

backward

ADD r1, r2, #4

SUB r1, r2, #4

ADD r4, r6, r7

Load and Store Instructions

Load and store instructions transfer data between memory and processor registers. They are three types of load-store instructions: single-register, multiple register and swap.

Syntax <LDR {str} {<cond>} {B} Rd, addressing
LDR {<cond>} SB|H|SH Rd, addressing.

LDR	Load word into a register	$Rd \leftarrow \text{mem}_{32}[\text{address}]$
STR	Store byte or word from a register	$Rd \rightarrow \text{mem}_{32}[\text{address}]$
LDRB	Load byte from a register	$Rd \leftarrow \text{mem}_8[\text{address}]$
STRB	Store byte from a register	$Rd \rightarrow \text{mem}_8[\text{address}]$

Example

PRE :

$r_0 = 0x00000000$

$r_1 = 0x00090000$

$\text{mem}_{32}[0x00090000] = 0x010101$

$\text{mem}_{32}[0x00090004] = 0x020202$

LDR $r_0, [r_1, \#4]$

Post

$r_0 = 0x020200202$

$r_1 = 0x00090004$

LDR $r_0, [r_1, \#4]$

$r_0 = 0x01010101$

2) Discuss the difference between the following ARM instructions with examples.

- ADD vs ADC
- MOV vs MVN
- CMP vs SUB

i) ADD vs ADC

SN	ADD	ADC
1.	Performs a simple addition of two operands.	Add two operands and the carry flag (C flag from CPSR)
2.	Does not consider the carry flag	Considers the carry flag
3.	When performing simple addition operations where the carry flag is not relevant.	When performing multi-precision arithmetic (eg. 64 bit addition using 32-bit registers) when the carry flag needs to be considered in the addition operation
4.	Eg: <code>ADD R0, R1, R2;</code> $R0 = R1 + R2$	Eg: <code>ADC R0, R1, R2</code> $R0 = R1 + R2 + \text{carry}$

(ii) MOV VS MVN

SNO	MOV	MVN
1.	Copies the value of an operand into a destination register unchanged	Moves the bitwise NOT (1's complement) of the operand into the destination
2.	Just moves a value or register content to another register.	It inverts all bits of the immediate or register sources.
3.	When the exact value needs to be copied without modification.	When working with binary data or bit level operations.
4.	Eg: <code>MOV R0, #0xFF;</code> $R0 = 0x000000FF$ <code>MOV R1, R0;</code> $R1 = R0$ (copies the value in $R0$ to $R1$)	Eg: <code>MVN R0, #0x00;</code> $R0 = 0xFFFFFFFF$ <code>MVN R1, #0xFF;</code> $R1 = 0xFFFFFFF0$

(iii) CMP VS SUB

S.No	CMP	SUB
1.	Perform subtraction, but does not store the result it updates condition flags only.	Subtracts one value from another and stores the result.
2.	The result of $R_1 - R_2$ is not saved, but the flags in the CPSR are updated.	Subtracts the value in R_2 from R_1 and stores the result.
3.	When comparing values and setting condition flags for conditional execution.	When performing subtraction operations and storing the result in a register.
4.	Eg. $CMP\ R_1, R_2$ Sets flags based on $R_1 - R_2$	Eg. $SUB\ R_0, R_1, R_2$ $R_0 = R_1 - R_2$

3. Describe different addressing modes supported by ARM architecture. Explain with suitable example.

- Immediate addressing.
- Register Indirect addressing.
- Pre-indexed addressing.
- Post-indexed addressing.

(i) Immediate addressing mode

→ The immediate addressing mode refers to the use of a constant value (immediate) directly within an instruction, rather than referencing it through a memory address or register.

→ This mode is common in data-processing instructions, where the operand is specified directly.

→ It is used in arithmetic, logical and more instructions.

→ It's fast because it doesn't require memory access or register lookup.

→ The immediate value is embedded in the instruction itself.

Example

1. MOV (Move Immediate value)

MOV R0, #5

→ This moves the immediate value 5 into register R0.

→ #5 is the immediate operand.

2. ADD (Add Immediate values)

ADD R1, R1, #10

→ This adds 10 to the contents of register R1 and stores the result in R1.

3. SUB (Subtract Immediate values)

SUB R2, R2, #3

→ Subtracts the immediate value 3 from register R2.

4. Compare [CMP] [compare with Immediate values]

CMP R3, #100

→ compares the value in R3 with the immediate value 100.

(ii) Register Indirect Addressing Modes

→ Register Indirect addressing is used when the memory address of the data is held in a register.

→ In ARM instructions, this mode is mainly used with load (LDR) and store (STR) instructions.

→ The register indirect addressing modes, which allow you to access memory locations using registers as pointers.

1. Register Indirect

Syntax [Rn]

Example - 1. DR, R0, [R1] (loads the value from the memory location pointed to by R1 into R0)

2. Register Indirect with offset

Syntax $[R_n, \#offset]$

Example $LDR R_0 [R_1, \#4]$ (loads the value from the memory location pointed to by $R_1 + 4$ into R_0)

3. Register Indirect with Register offset

Syntax: $[R_n, R_m]$

Example $LDR R_0 [R_1, R_2]$ (loads the value from the memory location pointed to by $R_1 + R_2$ into R_0)

4. Register Indirect with shifted Register offset

Syntax $[R_n, R_m, shift]$

Example $LDR R_0 [R_1, R_2, LSL \#2]$ (loads the value from the memory location pointed to by $R_1 + (R_2 \text{ shifted left by 2 bits})$ into R_0)

(ii) Pre-indexed addressing modes

→ Pre indexed addressing mode which allows you to access memory location using register as pointers and then update the point register.

→ It is a form of addressing where the effective memory address is calculated before the data transfer operation (load or store) and the updated address can optionally be written back to the base register.

Syntax

$LDR R_d [R_n, \#offset]!;$

Pre-indexed with write-back

$LDR R_d, [R_n, \#offset];$

Pre-indexed without write-back.

Examples

1. LDR with Pre-Indexed addressing (with write back)

LDR R1, [R0, #4]

Load value from memory at address $R_0 + 4$ into R_1 , and then update $R_0 = R_0 + 4$.

2. STR with Pre-indexed addressing (without write back)

STR R2, [R3, #-8]

Store the value in R_2 into memory at address $R_3 - 8$.

R_3 is not updated.

3. Using Register offset

LDR R4, [R5, R6]

Load value from address $R_5 + R_6$ into R_4 , and then update $R_5 = R_5 + R_6$.

Why use Pre-Indexed mode?

→ Efficient for accessing data structures like arrays, stacks and linked lists.

→ Reduces the numbers of instructions needed to update pointers.

(iv) Post-indexed addressing mode

→ Post indexed addressing mode is a form of addressing where the memory access (load/store) is done using the base register before the offset is applied, and then the offset is added or subtracted from the base register.

Syntax LDR Rd, [Rn], #offset;

Post indexed with immediate offset

STR Rd, [Rn], Rm

Post indexed with register offset.

→ The base registers can be updated with the offset after the memory operation.

Example

1. LDR with post-Indexed addressing

LDR R1, [R0], #4

Load value from memory at address R0 update $R0 = R0 + 4$

2. STR with post-Indexed addressing (register offset) :

STR R2, [R3], R4

Store value in R2 into memory at address R3, then update $R3 = R3 + R4$.

3. LDR with negative post-offset :

LDR R5, [R6], #8

Load from address in R6 into R5, then subtract 8 from R6.

4. Post Indexed with shifted register offset

LDR R0, [R1], R2, LSL #2

Load the value from the memory location pointed by R1 into R0 and updates R1 to $R1 + (R2 \text{ shifted left by } 2 \text{ bits})$

Why use post Indexed mode

- Reduces the need for additional arithmetic instruction to update the base register.
- Ideal for ~~iterating~~ iterating through memory structures where the pointer should move after accessing the data.

4. Differentiate between B, BL and BX instructions in ARM. how these instructions help in implementing functions call and returns.

S.No	B (Branch)	BL (Branch with Link)	BX (Branch and exchange)
1.	Causes an unconditional jump to the address specified by label.	Branches to the address specified by label, but also stores the return address (next instruction) in the link register.	Branches to the address stored in a register (commonly LR for function returns). Can switch between ARM and Thumb mode based on the LSB of the target address.
2.	Use case is used for jumps (branches) where the return is not needed. (eg: loops, if-else, jump, to error handler)	Use case is used for function calls, allowing the function to return to the caller using the stored address in LR.	Use case is used for function returns and mode switching in ARM/Thumb interworking.
3.	Effect is jumps to the specified label.	Effect is jump to the specified label and saves the return address in the link register (LR).	Effect is jumps to the address specified in the register Rm and exchange the instruction set (ARM or Thumb).
4.	No return address is saved.	The return address is the address of the instruction following the BL instruction.	The return address is not saved.
5.	<u>Syntax:</u> B label	<u>Syntax</u> BL label	<u>Syntax:</u> BX Rm BX (register)

- The B, BL and BX instructions are used for branching and function controlled in embedded systems.
- These instructions flow of the program, especially in function calls and returns.

Instruction helps in Implementing function call and Return

- This pattern makes B/BL/BX critical in implementing structural program control and function handling in ARM-based embedded systems.
- In embedded systems, these instructions are used to implement function calls and returns.

Function call

- When a function is called, the BL instruction is used to jump to the function's entry point. The return address is saved in the link register (LR).
 - Use BL function-label
 - The return address is saved in LR.
 - The BL instruction saves the return address in the link register (LR), eliminating the need for explicit stack management.
- The B instruction allows for unconditional branching, while the BX instruction allows for branching with instruction set exchange.
- Using these instructions can reduce code size and improve performance in embedded systems.

Function Return

- When a function returns, the BX LR instruction is used to jump back to the return address saved in the link register (LR).
 - Use BX LR instructions.

- Jumps back to the instruction after the BL
- This pattern makes B / BL / BX critical in implementing structural program and function handling in return function.
- Returns to the caller by branching to the address in LR.
- calls the double value function and saves the return address in LR.

Example

```

assembly
; Function call
BL my-function
; Function entry point
my-function:
; Function code
BX LR;
Return to caller,

```

- In this example, the BL instruction is used to call the (my-function) function.
- The return address is saved in the link register (LR)
- The function returns using the BX LR instruction, which jumps back to the return address saved in the link register (LR)